Mock B - Marking Scheme and Walkthrough

Module Score Logger: console CSV logger + pandas/numpy/matplotlib + SQLite. 40 marks. Model solution: `MockB_solution.py` *(tested with Python 3.14, pandas 3.1, numpy 2.5). Data:* `module_scores.csv` *(27 data rows) and* `stdin.txt` *(the scripted keyboard input).*

1. Marks overview and self-marking

Sub-task	Marks	Full marks need
1a	4	`os.path.exists`; header written only when missing; two different messages
1b	5	loop until `q`/`Q`; file opened in `"a"` mode; rows appended; `.title()` on names; "Data saved" printed
1c	5	`try/except ValueError`; score is int 0-100; hours float > 0; re-asks; empty name rejected
2a	3	`read_csv`, `drop_duplicates`, count of records + modules printed
2b	3	`np.mean/median/std/min/max` on the Score column, 2 d.p.
2c	2	text file written in `"w"` mode (so it never grows)
2d	4	one figure, two subplots; histogram + bar chart; titles/labels; `plt.show()`
3a	4	`connect("ScoresDB.db")`, `CREATE TABLE Results` with 4 columns and types
3b	4	rows inserted with `?`, `commit()`, duplicate-proof on re-run
3c	6	2 marks each: COUNT; AVG + GROUP BY + ORDER BY DESC; ORDER BY DESC + LIMIT 3

Style marks: names, comments, functions, tidy layout. Using functions (as the model solution does) is not required but reads well.

How to self-mark: sit the paper, run with the scripted input (`python your_file.py < stdin.txt`), then ALSO run it by hand once, then run it a second time typing only `q`. Compare with section 2.

2. Expected results (verified by running, and independently by a

plain-Python calculation with the `csv` and `statistics` modules)

Starting file: 27 data rows. After the scripted input, the CSV has 31 data rows (32 lines with the header): the 4 valid typed records are Mei, Dara, Kofi, Zoe. Console output of the later tasks:

```
Records read: 31, duplicates removed: 2
Total records: 29
Number of modules: 4

Total records: 29
Mean score:    70.93
Median score:  72.00
Std deviation: 14.73
Minimum score: 36
Maximum score: 97
Summary saved to score_summary.txt

Records in Results: 29
Average score per module:
  Chemistry  85.25
  Maths      70.38
  Biology    64.00
  Physics    61.50
Top 3 scores:
  Zoe      Chemistry  97
  Lena     Chemistry  95
  Lena     Maths      90
```

Module record counts: Chemistry 8, Maths 8, Physics 8, Biology 5. (Maths average is exactly 70.375, shown as 70.38.)

Idempotence check. Re-run with only `q` typed: Task 2 and Task 3 output is identical, and `score_summary.txt` has the same 6 lines. Re-run with the full `stdin.txt` again: the CSV gains 4 more lines each time (that is correct for a logger) but "Total records", every statistic and "Records in Results" stay at 29, because duplicates are removed in pandas and refused by the database.

Create path. Delete `module_scores.csv`, run, type `q`: the message says it was *created*; the file holds only the header; Task 2 then has 0 records (the model solution prints "No records yet" and stops cleanly; a crash here loses no marks if the 31-row case works, but handling it earns style credit).

3. Planted traps and how the solution handles them

Trap	Handling
Starter file already contains one exact duplicate row (Tara, Physics, 81, 6.8 twice)	`drop_duplicates()` in 2a; `UNIQUE` in the table
Scripted input re-types an existing record (Kofi, Biology, 72, 6)	same: 2 duplicates removed in total (31 -> 29)
Score `abc` (not a number) and `105` (out of range)	`int()` raises `ValueError` -> caught; range check fails -> message; ask again
Hours `-2` (negative) and `x` (text)	`float()` / `> 0` check; ask again
Empty student name	message, back to the name prompt (not saved)
`Q` in capitals; module typed in lower case	`name.lower() == "q"`; `.strip().title()`
Hours `6` typed without decimal	`float()` makes it 6.0, saved as `6.0` (matches the existing `6.0` rows, so the duplicate is detected)
Std deviation: numpy vs pandas	`np.std` divides by n (14.73); pandas `.std()` divides by n-1 (14.99). The paper says "population" so 14.73
Forgetting duplicates in stats	without `drop_duplicates` the mean would be 71.29 (31 rows), Biology 65.33, Physics 63.67
numpy ints into SQLite	`int(row["Score"])`, `float(row["Hours"])` (otherwise stored as BLOB bytes)
`open(..., "w")` instead of `"a"` for the logger	would wipe the 27 starter rows. Always "a" for append, "w" for the summary
Missing `newline=""` with `csv.writer`	on Windows adds blank lines between rows

TIP: `float("inf")` is accepted by `float()` and is `> 0`. The paper does not ask you to reject it; mention it in the viva if asked about robustness.

4. Walkthrough of the key lines

Task 1 - the logger (analogy: a guest book at a front desk)

```
if not os.path.exists(filename):                # is there a guest book?
    with open(filename, "w", newline="") as f:  # no: make a new one
        csv.writer(f).writerow(["Student", "Module", "Score", "Hours"])
    print(... "was created" ...)
else:
    print(... "already exists and is open for appending")
```

`"w"` means "start a fresh page" - which is exactly why it is only used when the file is missing. The `with` block closes the file for you, like a receptionist who always shuts the book after writing.

```
with open(filename, "a", newline="") as f:      # "a" = write at the end
    writer = csv.writer(f)
    name = input("Enter student name (or 'q' to quit): ").strip()
    while name.lower() != "q":                  # q or Q ends the loop
        ...                                     # ask, validate, writerow
        name = input("Enter student name (or 'q' to quit): ").strip()
```

This is the "ask once before the loop, ask again at the bottom" pattern, so `q` on the very first prompt skips the loop without a special case.

```
while True:                                     # repeat until good value
    try:
        score = int(input("Enter score (0–100): "))
        if 0 <= score <= 100:
            return score                        # good: leave function
    except ValueError:                          # 'abc', '7.5', '' ...
        pass
    print("Invalid score – please enter a whole number from 0 to 100.")
```

`try` = "attempt the risky step"; `except ValueError` = "if the typing cannot be turned into a number, jump here instead of crashing". Both failures (not a number, out of range) fall through to the same message.

Task 2 - pandas, numpy, matplotlib (analogy: spreadsheet + calculator

- chart wizard)

```
df = pd.read_csv(filename).drop_duplicates()    # table, photocopies out
scores = df["Score"].to_numpy()                 # one column -> numpy array
np.mean(scores); np.median(scores); np.std(scores)   # ddof=0 population
with open(summary_filename, "w") as f:          # "w": same file each run
    f.write("\n".join(lines) + "\n")
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))   # 2 side-by-side
ax1.hist(df["Score"], bins=6, edgecolor="black")        # left panel
ax2.bar(means.index, means.values)                      # right panel
```

`plt.subplots(1, 2)` = one sheet of paper with two frames. `ax1` / `ax2` are the frames: with subplots use `ax.set_title()`, `ax.set_xlabel()`, not `plt.title()`. `df.groupby("Module")["Score"].mean()` makes the bar heights.

Task 3 - SQLite (analogy: a filing cabinet with a strict clerk)

```
cursor.execute("CREATE TABLE IF NOT EXISTS Results (Student TEXT, "
               "Module TEXT, Score INTEGER, Hours REAL, "
               "UNIQUE (Student, Module, Score, Hours))")
cursor.execute("INSERT OR IGNORE INTO Results VALUES (?, ?, ?, ?)", ...)
conn.commit()
```

`IF NOT EXISTS` lets you re-run the file; `UNIQUE` + `OR IGNORE` refuse identical rows quietly; `commit()` saves the form (without it the changes vanish when the program ends).

```
SELECT COUNT(*) FROM Results;
SELECT Module, ROUND(AVG(Score), 2) AS AvgScore FROM Results
GROUP BY Module ORDER BY AvgScore DESC;
SELECT Student, Module, Score FROM Results ORDER BY Score DESC LIMIT 3;
```

`GROUP BY` = one pile per module, `AVG` = average of each pile, `ORDER BY ... DESC` = highest first, `LIMIT 3` = "only show the first three lines of the sorted list".

5. Viva-style follow-up questions

1. **Why do you open the logger file with `"a"` and the summary with `"w"`?** The logger must keep old records and add new ones at the end (append). The summary is a report that is rewritten completely each run, so write mode keeps it from growing.
2. **Why does `try/except ValueError` stop the crash, and what other check is still needed?** `int("abc")` raises `ValueError`, which we catch. A string like `105` converts fine, so the *range* (0-100) still needs an ordinary `if`.
3. **Why is `np.std` 14.73 but `df["Score"].std()` 14.99?** numpy divides by n (population) by default; pandas divides by n-1 (sample). Use `np.std(x, ddof=1)` to match pandas.
4. **How does your program avoid duplicate rows in the database when you run it three times?** `UNIQUE` over all four columns plus `INSERT OR IGNORE`; the pandas `drop_duplicates()` is a second layer (it removes them before they reach the database).
5. **What is the difference between `WHERE` and `HAVING`?** `WHERE` filters rows before grouping; `HAVING` filters groups after (e.g. `HAVING AVG(Score) > 65`).
6. **Why does the top-3 query use `ORDER BY Score DESC LIMIT 3`? What if two students tie for third place?** Sorting puts the largest first and `LIMIT` cuts the list. With a tie at third, only one of them appears (arbitrary); add a second sort key (`, Student`) to make it fixed, or query `Score >= third highest` to show both.
7. **Why call `conn.commit()`? What if you forget?** Inserts are held in a pending transaction; without `commit()` they are rolled back when the connection closes, and the table stays empty.
8. **How would you make the program accept `Q` or `q` without writing a record named "q"?** Compare with `name.lower() == "q"` *before* doing anything else with the name (the loop condition does this).
9. **What does `fig.tight_layout()` do?** Adjusts spacing so titles and axis labels of neighbouring subplots do not overlap.